

Dica

Rascunho isolado da Aula 6. A apostila completa (`gerar_apostila.py`) *não* muda. Quando este texto estiver pronto, ele entra no gerador principal.

Sumário

Clique no título para ir à seção.

AULA 6

Grafos e Algoritmos de Travessia 3

 O que é (e o que não é) 3

 Peças: vértice e aresta..... 4

 Não direcionado e direcionado..... 4

 Vértices adjacentes e grau 5

 Caminhos (e ciclo)..... 6

 Grafos ponderados 6

 Como guardar: lista ou matriz 7

 BFS com fila 8

AULA 6

Grafos e Algoritmos de Travessia

Objetivo desta aula (2h30)

Entender o grafo como modelo de relações (não como lista nem como fila), ler vocabulário (direção, adjacência, grau, caminho, peso), escolher como guardar e achar caminho com BFS.

Para seguir, use o VS Code

A instalação não entra no tempo de aula. Se ainda não tiver o programa, faça o passo a passo em

Instalação (pág. **Erro! Indicador não definido.**)

Se a tela falhar (Python não encontrado, pasta errada, NoneType), vá para **Se deu erro, faça isto** (pág. **Erro! Indicador não definido.**)

O que é (e o que não é)

Grafo *não* é lista, *não* é fila, *não* é deque e *não* é um jeito de ler uma lista. Grafo é o *modelo*: pontos e ligações. Serve quando o dado não cabe numa fila única — mapa, amizades, ruas, dependências.

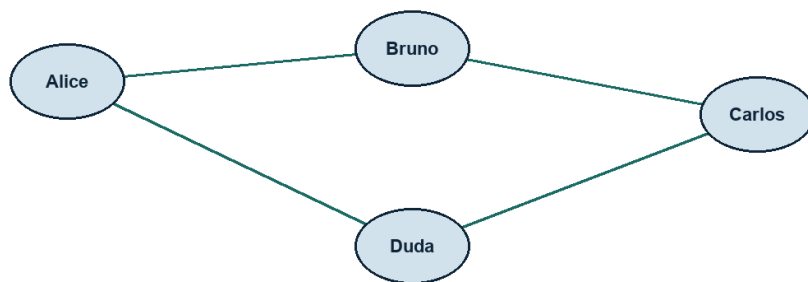
```
grafo          = o mapa (pontos + ligações)
lista / matriz = como o mapa é guardado na RAM
fila (deque)   = como a BFS anda no mapa
```

- ▶ *Grafo* — o mapa (quem se liga a quem).
- ▶ *Lista / matriz* — como esse mapa mora na memória.
- ▶ *Fila (deque)* — só a BFS usa, para visitar camada por camada.

Dica

Árvore é um grafo *sem ciclo* e com um começo (raiz). Grafo é o caso geral: pode ter ciclo (Alice–Bruno–Carlos–Alice) e mais de um caminho entre dois pontos.

Grafo: vértices (pessoas) e arestas (amizades)



BFS (fila): caminho mais curto em numero de arestas. Alice -> Carlos = Alice-Bruno-Carlos

Rede da prática: pessoas são vértices, amizades são arestas.

Peças: vértice e aresta

- ▶ *Vértice* (ou nó) — um ponto: pessoa, bairro, CD.
- ▶ *Aresta* — uma ligação entre dois vértices. No não direcionado escrevemos {Alice, Bruno}; no direcionado, CD → Centro.

No projeto final: o grafo é o *mapa*. Hash é a ficha do pacote. Heap é quem sai primeiro. Sem grafo você tem endereços soltos, sem rua entre eles — pergunta da Aula 8.

Não direcionado e direcionado

Não direcionado (o da aula e da logística simples): a ligação vale nos dois sentidos. Amizade, rua de mão dupla. Se A liga B, B liga A. Por isso o código faz *dois* `append`.

mão dupla

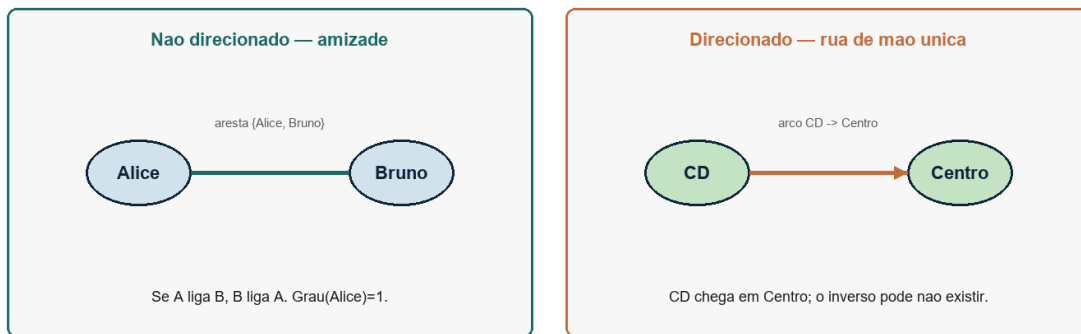
```
def adicionar_aresta(self, a, b):
    self.adicionar_vertice(a)
    self.adicionar_vertice(b)
    self.vizinhos[a].append(b)
    self.vizinhos[b].append(a)  # mao dupla
```

Direcionado: a ligação tem seta. Rua de mão única, “seguir” numa rede, voo Congonhas → Galeão. CD chega no Centro; o inverso pode não existir. Só um `append`.

mão única

```
def adicionar_arco(self, origem, destino):
    self.adicionar_vertice(origem)
    self.adicionar_vertice(destino)
    self.vizinhos[origem].append(destino)  # so um sentido
    # destino NÃO ganha origem
```

Nao direcionado (mao dupla) vs direcionado (seta)



Esquerda: aresta sem seta. Direita: arco com seta. No direcionado, grau de saída e grau de entrada separam.

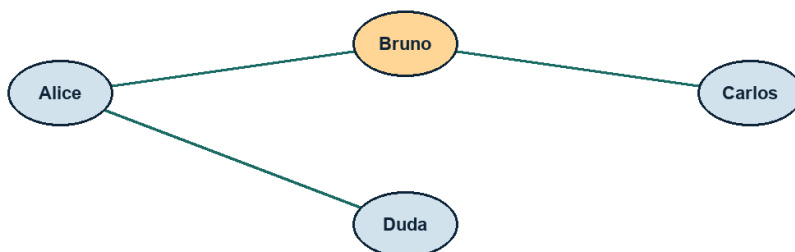
Atenção

O `06_grafo.py` do curso é *não direcionado*. Não copie o segundo `append` se a rua for de mão única — você inventaria o caminho de volta.

Vértices adjacentes e grau

Dois vértices são *adjacentes* (vizinhos) se existe aresta entre eles. *Grau* de um vértice = quantas arestas tocam nele. Na lista de adjacência, `grau(v) = len(self.vizinhos[v])`.

Adjacentes = vizinhos. Grau = quantas arestas tocam o vertice.



Bruno é adjacente a Alice e Carlos. Grau(Bruno)=2. Duda não é adjacente a Bruno.

Bruno é adjacente a Alice e Carlos. Grau(Bruno)=2. Duda não é vizinha de Bruno.

adjacência e grau na lista

```
vizinhos = {
    "Alice": ["Bruno", "Duda"],
    "Bruno": ["Alice", "Carlos"],
    "Carlos": ["Bruno"],
    "Duda": ["Alice"],
}
# Bruno é adjacente a Alice e Carlos
# grau("Bruno") = len(vizinhos["Bruno"]) → 2
```

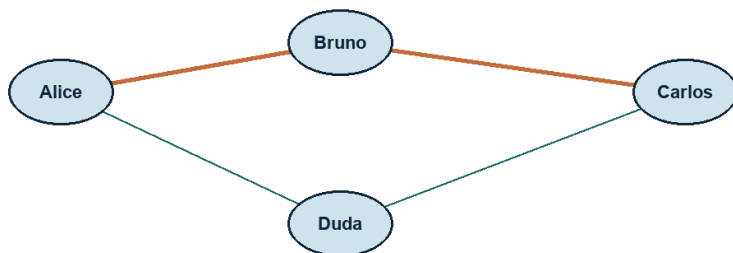
- ▶ Não direcionado: um número só (grau).
- ▶ Direcionado: *grau de saída* (quantas setas saem) e *grau de entrada* (quantas chegam).

- ▶ Folha da árvore da Aula 5 é o vértice de grau 1 (ou 0, a raiz sozinha) — mesma ideia, outro nome.

Caminhos (e ciclo)

Um *caminho* é uma sequência de vértices em que cada um é adjacente ao próximo. Neste curso o caminho *não repete* vértice. O *comprimento* é o número de arestas (Alice–Bruno–Carlos tem comprimento 2).

Caminho = sequência de vértices adjacentes (sem repetir, no curso)



Laranja: Alice-Bruno-Carlos (2 arestas). Teal: Alice-Duda-Carlos (também 2).

Ciclo: Alice-Bruno-Carlos-Duda-Alice. Árvore não tem ciclo; grafo pode ter.

Dois caminhos Alice → Carlos, ambos com 2 arestas. BFS devolve um deles, o de menos arestas.

- ▶ *Caminho* — vai de um ponto ao outro, sem repetir vértice.
- ▶ *Ciclo* — caminho que volta ao começo (Alice–Bruno–Carlos–Duda–Alice). Árvore não tem; grafo pode ter.
- ▶ *Conexo* (não direcionado) — de qualquer um dá para chegar em qualquer outro.

Dica

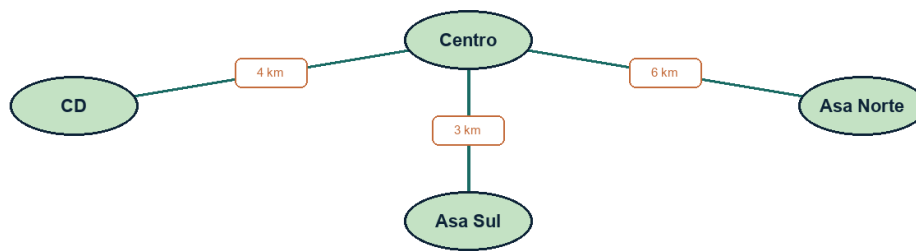
BFS (fila) acha o caminho com *menos arestas*. Se as ruas tiverem km diferentes, menos arestas ≠ menos km — aí o peso entra (mais abaixo).

Grafos ponderados

No grafo *não ponderado* da aula, toda aresta vale 1. No *ponderado*, a aresta carrega um custo: km, minutos, pedágio. Guardamos o par (*vizinho*, *peso*) na listinha.

aresta com km

```
def adicionar_aresta(self, a, b, km):
    self.vizinhos[a].append((b, km))
    self.vizinhos[b].append((a, km))
# CD --4km--> Centro  vira  ("Centro", 4) na listinha do CD
```

Grafo ponderado: a aresta carrega um custo (km, minutos, pedagio)

BFS ignora o km: conta arestas. Caminho mais curto em km pede outro algoritmo (depois do curso).

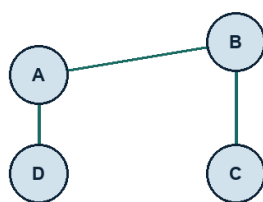
CD—Centro 4 km, Centro—Asa Sul 3 km. BFS ainda conta ruas, não soma km.

Atenção

A BFS desta aula *ignora* o peso. Caminho mais curto em km é outro algoritmo (Dijkstra) — fora do recorte de 2h30. No projeto, BFS no mapa não ponderado já vale.

Como guardar: lista ou matriz

O grafo é a ideia. Na RAM você escolhe uma *representação*. As duas guardam o *mesmo* mapa; mudam custo de memória e de consulta.

Mesmo grafo, duas representacoes**Lista de adjacencia**

A: [B, D]

B: [A, C]

C: [B]

D: [A]

Curso usa esta.

Grafo esparsos: poucas arestas.

Matriz V x V

	A	B	C	D
A	0	1	0	1
B	1	0	1	0
C	0	1	0	0
D	1	0	0	0

Olhar aresta é $O(1)$. Gasta $O(V^2)$.

Lista: para cada vértice, os vizinhos. Matriz: tabela $V \times V$ com 0/1 (ou o peso).

matriz do grafo A-B-C-D

```

#           Alice Bruno Carlos Duda
# Alice    0      1      0      1
# Bruno    1      0      1      0
# Carlos   0      1      0      0
# Duda     1      0      0      0
# 1 = tem aresta. Olhar a célula é  $O(1)$ . A tabela inteira gasta  $O(V^2)$ .
  
```

- *Lista de adjacência* — o curso. Boa quando o grafo é *esparso* (poucas ruas). Olhar os vizinhos de v é percorrer `vizinhos[v]`.

- ▶ *Matriz de adjacência* — tabela $V \times V$. “Existe aresta A–C?” é $O(1)$. Gasta $O(V^2)$ mesmo se quase tudo for zero.

Dica

`self.vizinhos` no código é um `dict` de listinhas — hash (Aula 4) apontando para listas (Aula 2). O *grafo* é o que isso representa, não o `dict` em si.

BFS com fila

Busca em largura visita camada por camada. A estrutura certa é a *fila* da Aula 3 (quem entra primeiro na visita sai primeiro). O caminho mais curto em *número de arestas* cai de graça se você guardar `veio_de`.

Passo 1. Enfileira o início. Marca `veio_de[inicio] = None`.

Passo 2. `popleft` no atual. Se for o destino, para.

Passo 3. Para cada *adjacente* ainda não visto: marca `veio_de[vizinho] = atual` e enfileira.

Passo 4. No fim, anda `veio_de` de trás para frente e inverte — esse é o caminho.

exemplos/06_grafo.py

```
from collections import deque

class Grafo:
    def __init__(self):
        self.vizinhos = {}

    def adicionar_vertice(self, nome):
        self.vizinhos.setdefault(nome, [])

    def adicionar_aresta(self, a, b):
        self.adicionar_vertice(a)
        self.adicionar_vertice(b)
        self.vizinhos[a].append(b)
        self.vizinhos[b].append(a)

    def bfs(self, inicio, destino):
        fila = deque([inicio])
        veio_de = {inicio: None}
        while fila:
            atual = fila.popleft()
            if atual == destino:
                break
            for vizinho in self.vizinhos[atual]:
                if vizinho not in veio_de:
                    veio_de[vizinho] = atual
                    fila.append(vizinho)
        if destino not in veio_de:
            return None
        caminho = []
```

```

    atual = destino
    while atual is not None:
        caminho.append(atual)
        atual = veio_de[atual]
    caminho.reverse()
    return caminho

rede = Grafo()
rede.adicionar_aresta("Alice", "Bruno")
rede.adicionar_aresta("Bruno", "Carlos")
rede.adicionar_aresta("Alice", "Duda")
print("Amigos de Bruno:", rede.vizinhos["Bruno"])
print("Caminho Alice → Carlos:", rede.bfs("Alice", "Carlos"))

```

Dica

Aqui o exemplo usa [deque](#) + [popleft\(\)](#) para a BFS ficar $O(1)$ na ponta da fila. É a mesma regra FIFO que vocês implementaram com nós. No projeto, pode colar a classe [Fila](#) da Aula 3 no lugar do deque. Vértices novos usam [setdefault\(\)](#). DFS usaria *pilha* — não garante menos arestas.

Práticas da aula

Prática A — No papel, o vocabulário

Desenhe 5 pessoas e algumas amizades. Marque: um par adjacente, o grau de cada um, um caminho de 2 arestas, um ciclo se houver. Diga se o desenho é direcionado ou não.

Prática B — Rede social restrita

Cadastre 5 pessoas e algumas amizades no [06_grafo.py](#). Mostre amigos em comum (interseção das listas) e o caminho mais curto entre dois usuários com BFS. Confira com o desenho.

Prática C — Mão única (no caderno)

Pegue CD → Centro e *não* desenhe a volta. Escreva a lista de adjacência. O que muda se alguém pedir o caminho Centro → CD?

Antes da próxima aula, você precisa conseguir

- ▶ Em uma frase: grafo ≠ lista ≠ fila. O que cada um faz nesta aula?
- ▶ Diferença não direcionado / direcionado no [append](#).
- ▶ O que é vértice adjacente e como lê o grau na lista.
- ▶ O que é caminho e por que ciclo existe no grafo e não na BST.
- ▶ Quando a matriz ganha da lista — e por que o curso usa lista.

- ▶ Por que BFS usa fila (e não pilha) para menos arestas.